Week 1 Workbook — Prince Buyinza (Setup Edition)

Track: Foundations + CI Discipline
Mentor: Paul Mwanje · **Repo (private):** `training-prince` · **Mirror (public):** `training-prince-mirror` (read-only)
Default branch: `development` (protected) · **Dates:** 10/09/2025

> Purpose: Prince sets up **everything himself** end-to-end so mentors can review daily with hard evidence (PRs, CI, artifacts). This mirrors John's updated workbook with clearer, step-by-step setup.

Objectives — What "Good" Looks Like by Friday

- End-to-end **GitHub flow**: feature branch → PR → **Quality Gate green** → review → merge to `development` → **mirror auto-updates**.
- **Three CLIs** shipped via PRs: **Hello → Stopwatch → Temperature Converter** (or To-Do, if time permits) with docs & tests.
- **Daily journals** contain timestamps and **evidence links** (PRs, CI runs, artifacts, screenshots).
- **Repo guardrails** in place: branch protection on `development`, required checks, PR template, labels, and minimal CI.

Pre-Flight (Day 0) — One-Time Setup

Goal: Create private source repo + public mirror and enforce guardrails so reviews rely on evidence, not opinion.

A) Repo Location & Naming

> You already created a private repo under your personal account. Move it into the org and rename it so we can apply guardrails and review in one place.

Option 1 — Transfer existing personal repo (preferred today)

1. Go to **GitHub → Your repo → Settings → General → Danger Zone → Transfer ownership**.
2. Type the repo name to confirm, choose destination ``.
3. Keep visibility **Private** (we will use a separate public mirror later).
4. After transfer, go to **Settings → General → Repository name** and rename to `` (or follow the naming you've been given).
5. Update your local git remote so pushes go to the new location:

```
git remote -v
git remote set-url origin git@github.com:Maximus-Technologies-Uganda/
```

```
training-prince.git
git fetch origin
```

6. Ensure you have access (push a tiny non-code change on a new branch and open a PR).
7. (If your repo already had a `main` default) create `development` and set it as default in the next section.

Option 2 — If no personal repo exists

1. Create **private** org repo `` (empty).
2. Create **public** org repo `` (empty; Actions disabled for now).

B) Default Branch & Protection (private repo)

1. Set default branch to ``.
2. Settings → Branches → **Add rule** for `development`:
3. ✅Require a pull request before merging (1 reviewer).
4. ✅Dismiss stale approvals.
5. ✅Require status checks → **checks** (CI) must pass.
6. ✅Restrict who can push (optional).

C) Labels (private repo)

Create: `needs-review-packet`, `Training`, `Review`, `Blocked`.

D) Minimal Project Files (initial commit)

Add these files:

```
README.md
.gitignore
package.json
.github/PULL_REQUEST_TEMPLATE.md
.github/workflows/checks.yml
/docs/journals/.gitkeep
/docs/workbooks/.gitkeep
/docs/review-packet-week1.md
/tests/.gitkeep
/src/hello/index.js
```

E) Node & Scripts

- Install Node **LTS**.
- In `package.json`, add scripts: `dev`, `test`, `lint` (any linter), `start` (for CLIs run via node).
- Pick a test runner (Vitest or Jest). Ensure `npm test` exits non-zero on failure.

F) CI — Minimal Quality Gate (private repo)

`.github/workflows/checks.yml` (runs on PR):

```
name: checks
on:
  pull_request:
    branches: [ development ]
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 'lts/*' }
      - run: npm ci
      - run: npm run lint --if-present
      - run: npm test
```

G) Mirror — Push-Only Automation (optional in Week 1)

Add a second workflow that **on push to **`` force-pushes to the public mirror using a repo-scoped token or deploy key. Keep trainees from touching the mirror directly. (You can enable this at the end of Week 1.)

H) Evidence Discipline (Daily)

Every day include: PR URL · CI run URL · Commit hash(es) · Artifact URL(s) · Time spent (hh\:mm) · Notes/Blockers · 1 screenshot.

Daily Plan (with Measurable Evidence)

> Follow the same rhythm every day. Keep PRs small (≤300 LOC), run tests locally before pushing, and capture evidence as you go—not at the end.

Daily Rhythm & Checklist (Mon–Fri)

Morning plan (09:00–09:20 EAT)

- Open today's journal file in `/docs/journals/YYYY-MM-DD.md`.
- Write 2–4 concrete tasks with time estimates. Link any open issues you'll work on.

Build blocks (09:20–12:30 & 14:00–16:30 EAT)

- Create a small **feature branch** from `development` for each task.
- Implement + write tests. Keep the CLI core in pure functions; keep I/O (reading args/printing) thin.
- Push early; open a **draft PR** once tests pass locally.

Midday check (12:30 EAT)

- Update the journal with: branch name, PR link, test count, blockers.

EOD hand-off (by 18:00 EAT)

- Convert draft PRs to **ready for review**. Ensure **checks** workflow is green.
- Update journal with PR/CI links, screenshot, and “What I learned”.
- Update the **Review Packet** with today’s PRs + CI snapshots.

Evidence you must capture daily
PR URL · CI run URL · commit SHA(s) · test count (today/total) · time spent · 1 screenshot · notes/blockers.

Day 1 — Bootstrap & Hello

Deliverables

- Node/Git configured; repo bootstrapped.
- "Hello, World" with **1 passing unit test** on PR; CI green.

Step-by-step

1. `git checkout -b chore/bootstrap` from `development`.
2. Initialize Node project (`npm init -y`). Add a test runner (**Vitest or Jest**).
3. Add scripts: `test`, `test:watch`, and (optional) `lint`.
4. Create `/src/hello/index.js` and `/tests/sanity.test.js` (always-pass test).
5. Commit & push; open PR → confirm `checks` runs and is green.

Definition of Done (DoD)

- PR open to `development` with template filled; 1 test passing locally & in CI; README explains install/run/test.

Day 2 — Hello CLI (Args & Flags)

Deliverables

- CLI accepts `--name` (or positional) and `--shout`.
- **2–3 tests** covering: name provided/missing; shout on/off.
- README: usage examples for all cases.

Step-by-step

1. `git checkout development && git pull` → `git checkout -b feat/hello-cli`.
2. Parse args (no heavy deps needed). Keep business logic in a pure function, e.g., `formatGreeting(name, shout)`; thin CLI wrapper reads `process.argv` and prints.
3. Write tests first for the pure function, then implement.
4. Update README with examples; run `npm test`; fix lint.
5. Push & open PR → ensure **checks** green.

DoD

- PR open; tests pass locally & in CI; README shows all usage; journal updated with PR/CI links + screenshot.

Day 3 — Stopwatch CLI (TDD)

Deliverables

- Commands: `start`, `lap`, `stop` → print total + laps.
- **≥3 tests**, incl. one **negative** (e.g., lap before start).

Step-by-step

1. `git checkout -b feat/stopwatch`.
2. Design first: split **core** (pure module: `start()`, `lap()`, `stop()`, `elapsedMs()`, `formatTime(ms)`) from **CLI** (arg parsing/printing).
3. Write tests for `formatTime` and lap behavior; include a negative test.
4. Implement core, then minimal CLI wrapper.
5. Push & open PR; verify **checks** are green.

DoD

- PR with ≥3 tests; green CI; journal includes failing→passing note for one test.

Day 4 — Temperature Converter CLI (Validation)

Deliverables

- Flags: `--from <C|F>` and `--to <C|F>`; reject illegal combos; clear error messages.
- **≥3 tests** covering valid conversions and error paths.
- README: examples incl. errors.

Step-by-step

1. `git checkout -b feat/temp-cli`.
2. Implement pure conversion functions and a validator; thin CLI wrapper maps flags to functions.
3. Add tests: C→F, F→C, and an invalid flag case.
4. Update README; push & open PR → **checks** green.

DoD

- PR open; ≥3 tests passing; README/examples updated; journal has PR/CI links.

Day 5 — Consolidation & Capstone PR

Deliverables

- Polish tests/docs across CLIs; optional tag `v0.1.0`.
- **Capstone PR** labeled `needs-review-packet` referencing `/docs/review-packet-week1.md`.

Step-by-step

1. `git checkout -b chore/week1-wrap`.
2. Tighten error messages, edge cases, and help text across CLIs.
3. Add/finish tests to reach **≥5 total** this week.
4. Update Review Packet with PR links, CI screenshots, and a 2–5 min demo video link.
5. Open **Capstone PR** to `development` → ensure **checks** green → add label `needs-review-packet`.

DoD

- Capstone PR ready; Review Packet complete; journal complete; (optional) release/tag created.

Acceptance Criteria (Pass Week 1)

- **Four PRs** total (bootstrap, hello-cli, stopwatch, temp) + capstone.
- **checks** workflow **green** on all PRs.
- Branch protection enabled on `development` (screenshot).
- Journals (Day 1–5) contain timestamps and **all evidence links**.
- (Optional) Mirror shows merged commit after `development` merge.

Templates (Copy-Paste)

PR Template — `.github/PULL_REQUEST_TEMPLATE.md`

```
## Summary
What & why (user impact)

## Scope
Issues/links

## How it works
Key decisions & tradeoffs; notable functions

## Screenshots / Demos
If applicable

## Test Plan
- [ ] Unit tests passing locally & in CI
```

```
- [ ] Manual steps: ...

## AI Usage
Prompts that materially changed your approach (summarize)

## Checklist
- [ ] Small, focused diff
- [ ] Lint/tests green
- [ ] README/docs updated
```

Review Packet — `/docs/review-packet-week1.md`

```
# Week 1 — Review Packet

## Scope & Links
- Bootstrap PR: ...
- Hello CLI PR: ...
- Stopwatch PR: ...
- Temp Converter PR: ...
- Capstone PR: ...

## CI Snapshots
- Links or screenshots to checks runs

## Diff Summary (3–7 bullets)
- ...

## Open Issues / Risks
- ...

## Rubric (100)
- Correctness (30): __/30
- Code Quality (20): __/20
- Tests (15): __/15
- Product Thinking (15): __/15
- CI Hygiene (10): __/10
- Docs/PR Notes (10): __/10

## Mentor Decision (Fri)
Promote / Hold / Remediate + notes
```

Daily Journal — `/docs/journals/YYYY-MM-DD.md`

```
# Journal — YYYY-MM-DD

## Goals for Today
- [ ] Task 1 (est. 60m)
- [ ] Task 2 (est. 90m)
```

```
## Prompts I used (5–10)
- "<prompt 1>" → key learning
- "<prompt 2>" → key learning

## Commands / Steps I tried
- `node src/hello --name John`
- `npm test`

## PRs & CI
- PR: <url> — CI: <url/screenshot>

## Bugs & Fixes
Symptom → root cause → fix

## What I learned (1–3 bullets)
- ...

## Plan for Tomorrow
- ...
```

Mentor Review Cadence (Daily)

1. Open Review Packet (if present) → skim PR links & CI
2. Fast rubric score (out of 100)
3. Write Mentor Note (3× good, 3× improve, ≤5 non-negotiables)
4. If off-track, open a blocking issue assigned to Prince

Quick Reference — Branch Protection UI Steps

1. Settings → Branches → **Add rule** for `development`
2. Require PRs, 1 reviewer, dismiss stale approvals
3. Require status checks → add `checks`
4. (Optional) Restrict who can push

Quick Reference — Evidence Discipline

Always include PR URL · CI run URL · commit SHA · artifact URL(s) · time spent · 1 screenshot.